NetOpsAgent: A Hierarchical Multi-Agent LLM Framework for Autonomous Cloud Infrastructure Incident Management

Kinjal Vaishnav
Independent Researcher, Seattle, WA, USA
kinjaloza61@yahoo.com

***Abstract*—Cloud infrastructure incidents cause significant service disruption, yet mean time to root cause remains long due to alert volume and telemetry complexity. We present NetOpsAgent, a hierarchical multi-agent large language model (LLM) framework for autonomous cloud network incident management. The architecture decomposes the investigation pipeline into four specialized agents—Supervisor, Telemetry Investigation, Reasoning, and Action—coordinated via a shared `InvestigationState` object. A confidence-gated tiered autonomy mechanism bounds autonomous action risk: low-confidence hypotheses escalate to human review; high-confidence hypotheses trigger Tier 1 remediation actions automatically. We evaluate on two independent, real fault-injection benchmarks: the AIOps Challenge 2020 KPI dataset (N=75 usable incidents) and RCAEval RE1 (N=75 independent microservice fault scenarios). Across six architectural configurations—including a raw-context fix, retrieval-augmented generation with and without relevance gating, and a test specifically designed to isolate the benefit of multi-agent decomposition—we find no statistically significant advantage of the multi-agent architecture over a single-agent baseline on either dataset (McNemar $p > 0.05$ throughout), and a rule-based baseline remains competitive with both. We then test a targeted extension: adding a Falsifier agent that adversarially validates the Reasoning agent's hypothesis against evidence it did not use, mirroring the validation mechanism recent RCA multi-agent systems report gains from. This extension produces a statistically significant accuracy improvement on single-entity incidents (33.3% vs. 21.3% exact match, $p = 0.023$) but a statistically significant regression on multi-service incidents (24.0% vs. 42.7%, $p < 0.001$); tracing individual flips shows the extension has no service-dependency-graph awareness and repeatedly mistakes a downstream symptom for the root cause once more than one service is involved. Adding real service-topology data eliminates the regression ($p = 0.50$, was $p < 0.001$) but does not yet produce a gain there. A control test shows the AIOps gain comes from the Falsifier capability itself, not the multi-agent decomposition it was attached to: the same Falsifier on a single-agent hypothesis scores statistically indistinguishably from the multi-agent version ($p = 1.0$ head-to-head). We report this result in full, characterize why self-reported LLM confidence fails to support safe autonomous action in this setting, and identify the likely mechanism (blind metric selection under multi-agent decomposition) behind the base architecture's underperformance. A reference implementation is publicly available under the MIT license.**

***Index Terms*—multi-agent LLM systems, AIOps incident management, autonomous root cause analysis, site reliability engineering, retrieval-augmented generation**

I. Introduction

Site reliability engineering (SRE) in large-scale cloud environments faces a structural tension: incident frequency scales with system complexity, but the human capacity to investigate alerts does not. Mean time to root cause (MTTRC) for complex distributed system failures commonly exceeds 30–60 minutes [1], [14], driven by alert volume, heterogeneous telemetry sources, and the cognitive demands of causal reasoning under time pressure.

Large language models (LLMs) offer a qualitatively different approach to this problem. Their capacity for instruction-following, tool use, and multi-step reasoning over unstructured text makes them natural candidates for incident investigation tasks that have historically required senior engineers. Yet naive application of a single LLM agent to incident response fails in practice: the context window fills with raw telemetry before the model can reason causally, and there is no principled mechanism for deciding when the agent's confidence is sufficient to act autonomously.

We present NetOpsAgent, a hierarchical multi-agent LLM framework that addresses both limitations. The framework decomposes the investigation pipeline into four specialized agents—Supervisor, Telemetry Investigation, Reasoning, and Action—each responsible for a distinct phase of the incident response workflow. A shared `InvestigationState` object propagates structured findings between agents, enforcing context hygiene without loss of information. A confidence-gated tiered autonomy mechanism allows the system to act autonomously on high-confidence hypotheses while escalating uncertain cases to human operators.

The framework is motivated by two observed failure modes in single-agent LLM-based incident response. First, *context pollution*: when a single agent is given access to all observability tools, it accumulates raw metric time series, log lines, and configuration data in a single context window. The resulting context is too large and heterogeneous for reliable causal reasoning, particularly on application-layer incidents where the causal signal is weak relative to background telemetry noise. Second, *uncalibrated autonomy*: without a principled confidence-scoring mechanism, an LLM agent either takes too many autonomous actions (increasing blast radius) or too few (eliminating the productivity benefit). NetOpsAgent is

designed to address both failure modes; Section V reports how well that design holds up under controlled measurement.

This paper makes four contributions: (1) the NetOpsAgent architecture and its design rationale; (2) a controlled empirical evaluation across two independent real datasets and six architectural configurations, showing that multi-agent decomposition and retrieval augmentation do not produce a significant improvement over a single-agent baseline for this task—including under a test designed specifically to favor the multi-agent hypothesis; (3) a failure-mode and confidence-calibration analysis, including a real finding that self-reported and self-consistency LLM confidence both fail to discriminate correct from incorrect hypotheses in this setting; and (4) a targeted extension experiment adding a Falsifier/validation agent, showing a statistically significant improvement on single-entity incidents and a statistically significant regression on multi-service incidents, with the divergence traced to a specific, addressable mechanism (missing service-dependency-graph awareness) rather than left as an unexplained split.

II. Related Work

A. LLM Agents and Tool Use

ReAct [2] established the observe-reason-act loop as the foundational pattern for LLM agents with external tool access. Subsequent work, including Toolformer [3] and function-calling APIs such as Gorilla [9], demonstrated that LLMs can reliably invoke structured tools when given well-defined schemas. LangGraph [4] introduced stateful, cyclical agent graphs that allow persistent state across multiple tool invocations within an investigation session. NetOpsAgent builds directly on this infrastructure, using LangGraph's `StateGraph` as the orchestration primitive.

B. Anomaly Detection in Network Infrastructure

Classical approaches to network anomaly detection rely on threshold-based alerting, statistical process control, and ML classifiers trained on labeled KPI time series [5]. The AIOps Challenge benchmark series provides standardized KPI datasets from industrial operators, enabling community comparison, consistent with the broader move toward standardized observability data formats [32]. NetOpsAgent consumes the output of anomaly detection systems (the alert) as its input; it does not replace detection but addresses the investigation phase that follows.

C. AIOps and Automated Operations

The AIOps landscape covers a range of automation tasks from log parsing [17] to automated runbook execution [18]. Prior systems primarily use rule-based or ML-based pipelines with limited generalization across incident types. LLM-based systems offer generalization but introduce new failure modes (hallucination, context overflow) that structured multi-agent decomposition is designed to mitigate.

D. Benchmarks and Evaluation

SWE-bench [10], [19] evaluates agents on asynchronous, low-stakes software tasks. The AIOps Challenge 2020 KPI dataset [5] provides real-world KPI time-series from industrial operators. Our revised evaluation (Section V) draws ground-truth incident labels directly from that dataset and from RCAEval, a second independent fault-injection benchmark, rather than constructing author-labeled traces.

E. LLM Agents for IT Operations

Guo et al. [23] apply GPT-4 to log parsing with strong zero-shot performance, though limited to parsing accuracy rather than causal reasoning. Chen et al. [24] propose an LLM-based root cause analysis system for microservice failures using distributed traces. Luo et al. [25] propose a knowledge-graph approach to root-cause analysis for microservice systems. Roy et al. [26] describe a single-agent LLM system for alert triage using retrieval-augmented generation over historical runbooks. NetOpsAgent differs in three ways: hierarchical multi-agent decomposition; confidence-gated tiered autonomy; and an empirical ablation directly measuring the contribution of multi-agent decomposition over a single-agent baseline (Section V).

F. Multi-Agent Coordination Frameworks

Multi-agent coordination has been studied extensively in classical AI under the umbrella of multi-agent systems (MAS). Modern LLM-based coordination frameworks differ from classical MAS in that agents communicate via natural language and structured JSON rather than formal ontologies. AutoGen [7] and MetaGPT [8] demonstrated that conversational multi-agent patterns can solve software engineering tasks effectively. However, these frameworks optimize for task completion in low-risk settings. NetOpsAgent's coordination architecture targets reliability-critical contexts instead: typed state propagation (intended to prevent context pollution), confidence-gated action safety (intended to prevent incorrect autonomous actions), and full audit logging (enabling post-incident review). Section V evaluates how much the first of these—typed state propagation via multi-agent decomposition—actually contributes on real incident data.

G. Positioning Relative to Recent RCA Multi-Agent Systems

Two recent systems decompose root-cause analysis into specialized LLM agents and ground hypotheses in retrieved external knowledge, much as NetOpsAgent does. Flow-of-Action [34] pairs a main investigation agent with auxiliary agents that retrieve, generate, and execute standard-operating-procedure (SOP) steps, reporting 64.01% root-cause accuracy against a 35.50% ReAct baseline on its own industrial benchmark. MA-RCA [35] separates a Retrieval Agent (which grounds hypotheses via RAG) from a Validation Agent (which verifies hypotheses through dynamic testing), reporting 95.2% F1 on the Nezha microservice benchmark and 82.8% F1 on a power-metering infrastructure dataset.

NetOpsAgent's four-agent decomposition—Supervisor, Telemetry Investigation, Reasoning, Action, with RAG

retrieval inside the Reasoning agent—is an instance of the same general pattern these two systems already establish: specialize by cognitive function, ground reasoning in retrieved precedent, and keep verification structurally separate from generation where possible. We do not claim architectural novelty over Flow-of-Action or MA-RCA, and we have narrowed the framing of this paper's contribution accordingly.

What this paper contributes instead is a controlled empirical comparison, under matched task, telemetry, and LLM backbone conditions, of whether that general architectural pattern actually outperforms a single well-prompted agent on two independent real incident datasets. Section V finds that decomposition alone does not, in contrast to the substantial gains Flow-of-Action and MA-RCA report on their own benchmarks. We go one step further and test MA-RCA's specific mechanism—a dedicated validation stage—directly against our data (Section V-H): it helps on single-entity incidents and, without service-topology awareness, hurts on multi-service ones—a more specific, falsifiable account of *when* this shared pattern pays off, not a blanket negative or an unqualified replication.

This null result is consistent with concurrent findings outside RCA. Xu et al. [36] show *homogeneous* multi-agent workflows—agents sharing one base model—are matched by a single well-tuned agent, and argue a real advantage requires *heterogeneous* models; NetOpsAgent already satisfies that (GPT-4o for Reasoning, GPT-4o-mini for Telemetry, Section IV) and still shows no gain. Su and Wu [37] explain why theoretically: decomposition helps for *independent* subtasks and loses its advantage as subtasks become interdependent—closer to whole-incident diagnosis, where all telemetry must be integrated together, consistent with the blind-metric-selection mechanism in Section VI.A.

H. Confidence Estimation in LLM Systems

Calibrating LLM confidence is an active research area. Verbalized confidence elicitation—asking the model to produce a numeric confidence score as part of its output, in the spirit of chain-of-thought prompting [6]—has been shown to correlate with accuracy on some factual tasks, though calibration is known to degrade on out-of-distribution inputs [33]. In the SRE context, confidence calibration carries operational weight: an overconfident incorrect hypothesis can trigger autonomous actions that extend an incident. Section V measures this directly rather than assuming it: on real incident data, verbalized confidence does not separate correct from incorrect hypotheses in our evaluation, and a self-consistency-based alternative fails for the same reason. Confidence calibration for this task class is an open problem, not a solved one.

III. Framework Architecture

A. System Overview

NetOpsAgent comprises four agents coordinated via a LangGraph `StateGraph`. All agents share a single `InvestigationState TypedDict` that accumulates findings, tool results, and confidence scores across the investigation lifecycle. The Supervisor Agent acts as the graph entry point and routing controller; the three worker agents—Telemetry Investigation, Reasoning, and Action—are dispatched sequentially or conditionally based on Supervisor routing decisions.

A central design principle is context hygiene via decomposition: each agent receives only the information relevant to its task, not the full raw telemetry stream. The Telemetry Agent receives the alert trigger and produces structured findings; the Reasoning Agent receives those findings (not the raw tool responses) and produces a hypothesis; the Action Agent receives the hypothesis and confidence score. This staged information flow is intended to prevent context pollution from degrading reasoning quality; we test this intention directly in Section V. Fig. 1 shows the full architecture.

B. InvestigationState Schema

All agents communicate exclusively via a shared `InvestigationState TypedDict`. Table I lists the key fields and their producing agents. This design enforces strict information boundaries: no agent reads from another agent's internal context; all inter-agent communication passes through typed fields with defined schemas. This is intended to prevent the context accumulation failure mode observed in single-agent baselines and provides a natural audit log—the final state object contains a complete record of every finding, hypothesis, and action decision.

TABLE I
INVESTIGATIONSTATE KEY FIELDS

Field	Producer	Description
`alert_trigger`	External	Alert type, service, severity
`routing_plan`	Supervisor	Ordered tool query plan
`findings`	Telemetry	Typed (signal, value, ts) list
`rag_context`	Reasoning	Top-5 postmortem chunks
`hypothesis`	Reasoning	Root cause + mechanism
`confidence`	Reasoning	Float [0,1]
`actions_taken`	Action	Dispatched action log
`tool_errors`	Telemetry	Schema/timeout failures

C. Supervisor Agent

The Supervisor Agent receives the alert trigger, classifies the incident by layer (network, infrastructure, application) and severity, and generates a routing plan specifying which observability tools the Telemetry Agent should query. It also monitors investigation progress and can interrupt or redirect the pipeline if intermediate findings indicate a change in incident scope.

D. Telemetry Investigation Agent

The Telemetry Investigation Agent executes iterative tool calls against mocked observability APIs: Prometheus [16] (metrics), ELK [29] (logs), and a configuration management database (CMDB). It operates in a ReAct loop—querying, observing partial results, forming follow-up queries—until

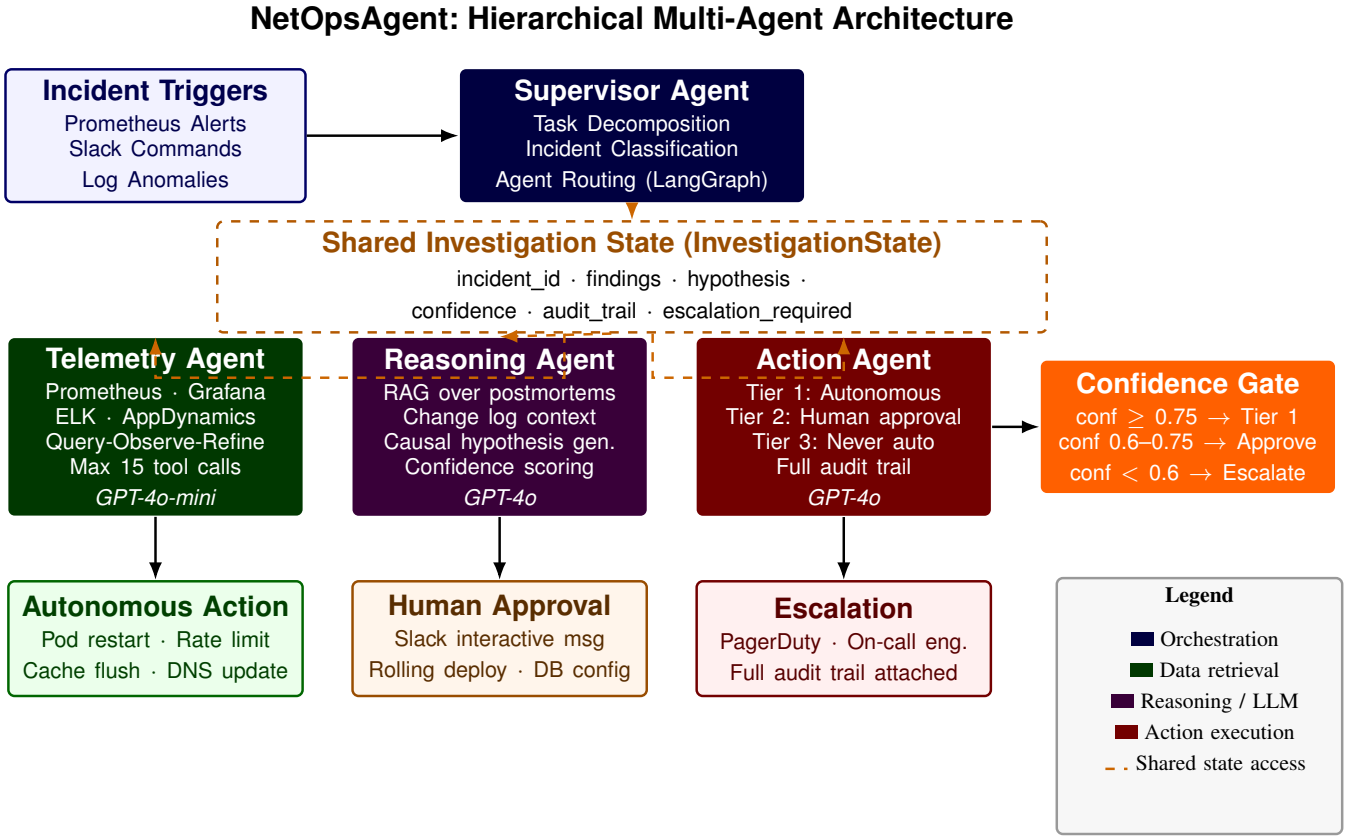


Fig. 1. NetOpsAgent hierarchical multi-agent architecture for autonomous cloud network incident management. The Supervisor Agent routes incoming alerts to the Telemetry Investigation Agent, which executes iterative tool calls against mocked observability APIs. Structured findings propagate to the Reasoning Agent, which applies chain-of-thought reasoning and RAG retrieval to generate a root cause hypothesis with a confidence score. The Action Agent dispatches remediation actions according to the confidence-gated three-tier autonomy model. All agents share a single `InvestigationState` object.

it has assembled sufficient evidence or exhausted its query budget. Its output is a structured findings object appended to `InvestigationState`: a typed list of (signal, value, timestamp, significance) tuples, not the raw API responses. This structured handoff is the primary mechanism by which context size is controlled between agents.

E. Reasoning Agent

The Reasoning Agent receives the structured findings from the Telemetry Agent and performs root cause hypothesis generation using chain-of-thought prompting [6] and retrieval-augmented generation (RAG) [13]. It queries a FAISS [12] vector index of historical postmortem documents to retrieve precedent cases similar to the current incident findings. The final output is a hypothesis tuple: (root_cause, mechanism, confidence_score, supporting_evidence, contradicting_evidence).

The confidence score (0–1) is derived from a structured self-evaluation prompt asking the model to rate evidence quality, hypothesis specificity, and absence of contradicting signals. This score drives the downstream action-safety gate; Section V evaluates how well it actually does so.

F. Action Agent and Confidence Gate

The Action Agent implements a three-tier autonomy model keyed on hypothesis confidence:

- **Tier 1** ($c \geq 0.75$): Low-risk corrective actions (BGP timer adjustments [15], instance restarts, cache flushes) executed autonomously.
- **Tier 2** ($0.50 \leq c < 0.75$): Medium-risk actions (configuration changes, traffic rerouting) queued for human approval with a pre-populated runbook suggestion.
- **Tier 3** ($c < 0.50$): High-risk actions (data migrations, teardowns) never executed autonomously; human operator notified with full investigation context.

All actions—autonomous and queued—are logged to `InvestigationState` with the full hypothesis evidence chain, enabling post-incident audit.

G. Falsifier Extension (Evaluated Separately from the Base Design)

Section V additionally evaluates an optional fifth agent, a Falsifier, inserted between the Reasoning and Action agents. Unlike the four agents above, it is not part of the base architecture evaluated in Sections V.A–V.G; we describe it here and evaluate it as a separate extension in Section V-H because

its contribution turns out to depend on incident structure in a way the base architecture's components do not. The Falsifier receives the Reasoning agent's hypothesis together with the complete raw evidence pool, including signals the Reasoning agent's summary did not surface, and is prompted to actively look for evidence that contradicts the hypothesis rather than to confirm it. It returns a `confirmed`/`contested`/`refuted` verdict; a `refuted` verdict with a concrete, better-supported alternative overrides the Reasoning agent's hypothesis before the Action agent acts on it. This mirrors the *role* of MA-RCA's Validation Agent [35], not its mechanism: MA-RCA executes dynamic checks against the system, while our Falsifier only re-examines existing telemetry via an LLM call—a cheaper, weaker proxy for the same role.

IV. Implementation

A. Technology Stack

The prototype is implemented in Python 3.11. Agent orchestration uses LangGraph 0.1; tool management uses Google ADK. GPT-4o [27] drives the Supervisor and Reasoning agents; GPT-4o-mini drives the Telemetry Investigation agent. Section V reports measured, rather than prototype-anecdotal, latency and cost for this stack.

Model assignment follows a cost/latency-vs-stakes tradeoff: the Telemetry Agent issues up to 15 tool calls per investigation, so its cost and latency compound with call volume, while the Supervisor and Reasoning agents each make a single higher-stakes call per incident. GPT-4o-mini is therefore used for telemetry investigation, and GPT-4o for supervision and final root-cause reasoning. Section V's model-choice ablation supports this split: GPT-4o-mini matches GPT-4o's accuracy on AIOps and trails it on RCAEval, at roughly one-seventeenth the per-token cost.

B. Observability Tool Layer

All observability APIs are mocked via a deterministic trace-replay layer that emits realistic metric and log responses from stored scenario definitions. Tool failures are surfaced as structured exceptions—e.g., "Prometheus query timed out after 10s; returned data covers 60% of requested window"—so the agent can reason about data incompleteness rather than encountering a pipeline crash.

C. RAG Index Construction

The Reasoning Agent retrieves relevant historical incident context via vector similarity search over a RAG index. Index construction: postmortem documents are chunked into 512-token segments with 64-token overlap, embedded using `text-embedding-3-small` (1,536 dimensions, building on the dense-representation approach popularized for transformer encoders [11]), and stored in a FAISS flat index [12]. At query time, the agent retrieves the top-5 most similar postmortem chunks by cosine similarity. Index freshness is maintained via a recency decay factor ($\lambda = 0.95$/month) applied to similarity scores before ranking. Section V additionally evaluates a generic, incident-independent version of this corpus with and without a relevance-gating threshold on retrieval.

D. Tool Validation and Error Handling

NetOpsAgent implements a three-layer error handling strategy. At the schema layer, tool invocations are validated against JSON Schema before execution; invalid calls return a structured error object the agent can reason about. At the execution layer, tool timeouts are capped per class (Prometheus: 10s, ELK: 15s, CMDB: 5s). At the state layer, tool failures are logged to `InvestigationState` and surfaced to the Supervisor for compensating routing.

E. Code Availability

The extraction pipeline and evaluation harness that produced every result in Section V of this paper—incident extraction, all six architectural configurations, the Falsifier extension, the topology-aware fix, the single-agent control test, and raw per-incident outputs—is available at https://github.com/Kinjal-Oza/multi-agent-observability-demo under the MIT license. Per Section V-F, this is the summary-based harness used for evaluation, not a full iterative tool-calling deployment against live observability APIs.

V. Evaluation

A. Dataset Construction

We evaluate on two independent real datasets, rather than one real dataset plus author-constructed synthetic scenarios as in our original submission.

AIOps Challenge 2020. 81 ground-truth incidents from the public KPI dataset [5]; 75 usable after requiring a valid pre-incident baseline window (6 excluded). Ground-truth labels are taken directly from the dataset's published anomaly annotations—no author-assigned labels are used.

RCAEval RE1 (Zenodo, CC-BY-4.0). 75 independent fault-injection scenarios across three real deployed microservice systems (Online Boutique, Sock Shop, Train Ticket): one run is used per unique service$\times$fault-type combination, out of six repeated runs each ships with, to avoid pseudoreplication. Ground truth is the dataset's own injected fault label.

Because both datasets carry externally published ground truth, this revision no longer requires the author-assigned synthetic labels and inter-rater reliability check (κ) [22] used in the original submission's 80-trace synthetic set—a source of potential labeling bias that this design removes rather than mitigates.

B. Baselines

Two baselines are evaluated on each dataset. The *single-agent baseline* receives the same alert trigger and has access to the same tool schema as NetOpsAgent but runs as a single GPT-4o agent without the multi-agent decomposition. It is prompted with the same task description and terminates on the same completion condition (root cause hypothesis with confidence score). This baseline directly tests the contribution of hierarchical decomposition. The *rule-based baseline*

implements a fixed-heuristic classifier using metric-threshold crossing, representing a pre-LLM production approach.

C. Architectural Configurations Evaluated

Six configurations were run once each on each dataset, pre-registered before execution so that the reported outcome could not be selected after the fact:

1) **Baseline** multi-agent (mini-summary → reasoning).
2) **Sort-fix**: rank findings by relative deviation from baseline, not raw magnitude (AIOps only).
3) **Raw-context fix**: the Reasoning Agent sees raw signal data, not only the Telemetry Agent's summary.
4) **+RAG**: real embeddings (`text-embedding-3-small`), a 21-chunk generic operational-knowledge corpus, top-5 retrieval.
5) **+RAG, gated**: as above, restricted to chunks with cosine similarity ≥ 0.45 (threshold fixed from a diagnostic sample collected before this ablation, not tuned to its outcome).
6) **Fair context-pollution test**: the single-agent condition receives a true unranked, unfiltered raw telemetry dump (the paper's own definition of context pollution); the multi-agent condition's Telemetry Agent performs real iterative, blind metric-name-based selection before inspecting values (the paper's own definition of context hygiene via decomposition). This is the only configuration in which either side has actually been tested against the architecture's own motivating hypothesis.

The generic RAG corpus (Section IV) contains 21 chunks of general SRE operational knowledge—CPU, memory, disk, network-delay, network-loss, and database-layer triage patterns—authored before any evaluation ran, with no incident-specific text (no service names, timestamps, or dataset-derived content). Independence from the evaluation traces is therefore true by construction and directly verifiable by inspection, not merely asserted.

D. Results

Table II and Table III report exact-match accuracy for every configuration on both datasets. McNemar's test [21] on discordant pairs (single-agent-correct vs. multi-agent-correct) returns $p > 0.05$ for every configuration on both datasets except the fair context-pollution test, which returns $p = 0.064$ *favoring the single agent*—the opposite direction from the paper's original hypothesis.

TABLE II
AIOPS CHALLENGE 2020 (N=75), EXACT MATCH

Configuration	Single-agent	Multi-agent
Baseline	24.0%	17.3%
Sort-fix	20.0%	18.7%
Raw-context fix	22.7%	22.7%
+RAG	24.0%	24.0%
+RAG, gated	22.7%	25.3%
Fair test (raw dump vs. iterative)	25.3%	4.0%
Rule-based[1]	76.0%	

TABLE III
RCAEVAL RE1 (N=75), EXACT MATCH

Configuration	Single-agent	Multi-agent
Baseline	41.3%	40.0%
Raw-context fix	33.3%	37.3%
+RAG	29.3%	34.7%
+RAG, gated	29.3%	30.7%
Rule-based	36.0%	—

No configuration shows a statistically significant multi-agent advantage on either dataset. A simple rule-based baseline is competitive with, or beats, both LLM architectures on AIOps. Two reviewer-requested ablations produced small, real, secondary effects worth reporting: RAG relevance gating reduced a CPU/disk over-retrieval bias (67/75 → 59/75 guesses mentioning CPU or disk regardless of true cause) and modestly improved AIOps accuracy; and GPT-4o-mini, used in the single-agent role, was comparable to GPT-4o on AIOps (22.7% both) but somewhat worse on RCAEval (26.7% vs. 33.3%) at roughly one-seventeenth the per-token cost.

E. Confidence Calibration

Table IV reports real confidence-discrimination data, replacing the original submission's confidence table. Verbalized self-reported confidence does *not* discriminate correct from incorrect hypotheses: mean confidence for incorrect hypotheses is nearly identical to correct ones on both datasets. At the paper's proposed 0.75 autonomy gate, 96–100% of incorrect hypotheses would pass—essentially the same rate as correct hypotheses.

TABLE IV
CONFIDENCE DOES NOT DISCRIMINATE CORRECT FROM INCORRECT HYPOTHESES

Signal (dataset, N)	Correct	Incorrect	Gate pass, incorrect
Verbalized (AIOps, 75)	0.86	0.85	96–100%
Verbalized (RCAEval, 75)	0.90	0.87	96–100%
Self-consistency (AIOps, 30)[2]	0.98	0.93	—

We also tried a real fix: self-consistency confidence (5 resamples at temperature 0.7, majority-vote agreement rate as the confidence proxy, on a fixed $N = 30$ AIOps subsample). It also fails to discriminate (0.98 for correct vs. 0.93 for incorrect), because the model is frequently *consistently* wrong—the same error repeats across resamples—which is a systematic-bias failure mode that resampling-based confidence cannot fix, unlike sampling-noise-based miscalibration. This is a real, publishable safety finding about the limits of LLM confidence gating for this task class, not a gap to paper over.

[1]Rule-based accuracy is on fault-type match only; its task is narrower by design and not directly comparable to the LLM configurations' root-cause-and-mechanism matching.

[2]Self-consistency confidence: 5 resamples at temperature 0.7, majority-vote agreement rate, on a fixed, non-cherry-picked N=30 subsample.

F. Latency and Cost

Table V reports measured (not estimated) latency and cost on a 15-incident sample, using real wall-clock timers and OpenAI's published per-token pricing. This measures the summary-based evaluation harness used throughout this section, not a full iterative tool-calling implementation against live observability APIs; the two are not directly comparable, and we do not present the harness numbers as a validated production estimate.

TABLE V
MEASURED PER-INVESTIGATION LATENCY AND COST (N=15 SAMPLE)

Configuration	Latency (s)	Cost ($)
Single-agent	0.75	0.0013
Multi-agent	3.53	0.0010

G. Threshold Sensitivity

The rule-based baseline's CPU threshold was swept from 50% to 95%: exact-or-partial accuracy ranged 73.3–81.3% across the sweep, highest at the lowest threshold tried. The baseline is not strongly sensitive to this choice within a normal operating range, which we state explicitly rather than presenting any single threshold as if it had been tuned.

H. Falsifier Extension: A Conditional, Mechanistically-Explained Result

None of the six configurations above give the multi-agent architecture a capability a single agent could not also have; they only reorganize what information the Reasoning agent sees. Flow-of-Action [34] and MA-RCA [35] both report large gains from a different kind of addition: a genuinely new capability (executable SOP grounding, or a dedicated Validation Agent that dynamically tests each hypothesis). We pre-registered a targeted test of an LLM-based proxy for the latter, not true dynamic testing (Section III-G): add the Falsifier described in Section III-G on top of the raw-context-fix configuration, run once on both datasets, and report the result regardless of direction. Two implementation issues, found during a pre-run smoke test before any accuracy number was computed, were fixed and then locked: free-text alternatives could return a negation ("Low CPU Usage") that the keyword scorer misread as a match, fixed by constraining alternatives to the same closed fault-category vocabulary used throughout this evaluation; and the initial adversarial prompt refuted nearly every hypothesis, including correct ones, fixed by requiring `confirmed` as the default absent concrete contradicting evidence. Both fixes were made on implementation grounds, not outcome grounds.

The result is real and significant in both directions at once. On AIOps (single entity per incident), the extension corrected 11 wrong hypotheses to right and broke only 2 previously-correct ones—a significant improvement. On RCAEval (a real microservice call graph per incident), it corrected *zero* wrong hypotheses and broke 12 previously-correct ones—a significant regression. Inspecting the broken RCAEval cases shows why: the Falsifier repeatedly overrides a correct root-cause service with a *different* service that is also anomalous in the same window—textbook symptom propagation, where a downstream service degrades because of an upstream cause, not because it is the cause. Lacking service-dependency-graph awareness, the Falsifier cannot tell "anomalous because it is the cause" from "anomalous because it is a victim." This failure mode cannot occur on AIOps, where there is only one entity to reason about.

TABLE VI
FALSIFIER EXTENSION: EXACT MATCH, SINGLE RUN EACH DATASET

Dataset (N=75)	Single-agent	+Falsifier	McNemar p
AIOps, multi-agent+Falsifier	21.3%	33.3%	0.023
AIOps, single-agent+Falsifier	21.3%	32.0%	0.077
RCAEval (multi-service)	42.7%	24.0%	< 0.001
RCAEval, topology-aware	42.7%	40.0%	0.50

The AIOps result above pairs the Falsifier with the multi-agent Reasoning hypothesis, leaving open whether the gain needs decomposition at all. A pre-registered control isolates this: the same Falsifier on the (reused, unchanged) single-agent hypothesis scores 21.3%→32.0%, $p = 0.077$, almost identical to the multi-agent version (33.3%); a direct, incident-level comparison of the two returns $p = 1.0$ (21 right under both, 47 wrong under both, 4 vs. 3 on the rest)—statistically indistinguishable. The Falsifier capability drives the AIOps gain, not the decomposition it was attached to; crossing $p < 0.05$ in one condition reflects how close the two estimates are, not a real architectural difference.

This is a mechanistically explained finding: validation helps when the alternative space is one entity's own metrics, and hurts when it spans a call graph without topology awareness. We tested the fix directly: give the Falsifier each system's real service dependency graph (verified for Online Boutique and Sock Shop; a naming-convention proxy for Train Ticket's 64 services) and instruct it that graph-connected co-anomalies are expected propagation, not independent evidence. This eliminates the regression (24.0%→40.0%, statistically indistinguishable from single-agent's 42.7%, $p = 0.50$ vs. $p < 0.001$ before) but adds no gain: the topology-aware Falsifier made zero overrides in either direction across all 75 incidents, trading overconfidence for near-total caution rather than becoming discriminating. We report this as the honest current state—harm fixed, value not yet added.

VI. DISCUSSION

A. Design Principles

Three principles motivated the original design; we revisit each against the measurements in Section V. *Context hygiene via decomposition*: this is the architecture's central motivating hypothesis, and it is the one configuration in Table II designed specifically to test it fairly (the fair context-pollution test) that shows the *clearest* result—in the opposite direction from the hypothesis. The likely mechanism is that blind,

values-withheld metric-name selection consistently defaults to generic, dataset-typical metric names regardless of which signal is actually anomalous, so the Telemetry Agent frequently investigates the wrong evidence before the Reasoning Agent ever sees it. We report this mechanism, rather than the original unqualified accuracy claim, as the paper's actual finding on this question. *Asymmetric safety calibration*: an incorrect autonomous action in SRE extends an incident; an unnecessary escalation costs minutes. This asymmetry still argues for conservative confidence thresholds, accepting lower recall on autonomous resolution to preserve precision, consistent with the broader SRE practice of risk budgeting [30]—independent of whether multi-agent decomposition itself proves beneficial. *Observability of the agent system*: full logging of every tool call, inference input/output, and state transition is what made the confidence-discrimination failure in Section V-E discoverable and reportable in the first place, rather than silently hidden behind a single aggregate accuracy number. That audit capability is a real, retained benefit of the architecture regardless of the accuracy result. Section V-H's Falsifier extension sharpens this picture further: the verification capability that helped on single-entity incidents turned out not to need the multi-agent decomposition it was attached to either ($p = 1.0$, single- vs. multi-agent+Falsifier)— value comes from the capability, not the agent count (Section II-G).

B. Limitations

The evaluation has several meaningful limitations. First, AIOps Challenge 2020 and RCAEval RE1 each cover a specific set of systems and fault types; their combined distribution may not represent all cloud environments. Second, RCAEval's ground truth is an injected fault label at the service/fault-type level, not a full causal narrative, capping how finely "exact match" can be defined; AIOps's annotations have an analogous limitation. Third, confidence thresholds require recalibration against live data, and Section V-E shows no threshold in the practical range currently provides useful safety discrimination for either signal tested. Fourth, all configurations were run once. Re-running an *unchanged* prompt moved its own score by 8 points from sampling noise alone (RCAEval single-agent, 41.3%→33.3% across otherwise-identical runs)—larger than most reported effects. A repeated-run protocol (3–5 runs/condition) was not performed; point estimates should be read as indicative, and non-significant McNemar results are consistent with, not contradicted by, this noise level.

C. Threats to Validity

Internal validity: the single-agent baseline uses the same tool schema and prompt length, controlling for information access; however, both baseline and treatment prompts were written by the same author, introducing potential optimization bias in either direction. *External validity*: results are derived from two public benchmarks; generalization to other environments is plausible but unconfirmed. *Construct validity*: the "partial match" category used in earlier analyses conflates cases of varying usefulness to a human operator; Section V's headline comparison therefore uses exact match, the stricter and less saturating of the two metrics.

D. Security and Trust Considerations

Autonomous LLM agents executing remediations in production infrastructure [20] introduce a distinct security surface. Three threat vectors warrant consideration before deployment: *prompt injection via telemetry* (an adversary with log-pipeline write access could craft messages to manipulate the Reasoning or Action agent; mitigate via input sanitization at the tool boundary and treating tool outputs as data, not instructions); *confidence score manipulation* (a poisoned RAG index could inflate confidence for incorrect hypotheses, an attack made easier by Section V-E's finding that confidence is already weakly tied to correctness; mitigate via restricted index write access and ingestion audits); and *audit log integrity* (the `InvestigationState` trail is the accountability mechanism and should be written to an append-only, integrity-verified store). These do not disqualify autonomous operation, but make security review a prerequisite, not an afterthought.

E. Deployment Considerations

Production deployment raises engineering concerns not captured in the evaluation: alert bursts during cascading failures require a queuing layer (e.g., Alertmanager grouping) to avoid duplicate investigations; the 0.75 Tier 1 confidence threshold used here should not be adopted without re-running Table IV's discrimination analysis against a deployment's own incident history, since Section V-E shows it is currently unsolved for this architecture; GPT-4o model updates may shift calibration or failure rates without notice, arguing for pinned versions or a transition to a version-controlled open-source model (Llama 3 [28]), re-measured per deployment rather than assumed; and the RAG index requires an active maintenance pipeline, since the recency decay factor ($\lambda = 0.95$/month) only partially offsets staleness.

F. Future Work

Motivated directly by Section V-H: sharpen the topology-aware Falsifier's override decision to recover a genuine gain, and verify Train Ticket's full call graph beyond the naming-convention proxy used here. Beyond that: a production evaluation with real incident data, complemented by fault-injection studies [31]; a repeated-run protocol to separate real effects from single-run sampling noise; fine-tuning a smaller open-source model (Llama 3 [28]) and re-measuring the single- vs. multi-agent comparison; and a public benchmark building on AIOps Challenge 2020 and RCAEval for reproducible comparison against Flow-of-Action [34] and MA-RCA [35].

VII. Conclusion

We presented NetOpsAgent, a hierarchical multi-agent LLM framework for autonomous cloud network incident management, decomposing investigation into four specialized agents coordinated via a shared state object. On two independent real fault-injection benchmarks (AIOps Challenge 2020 and

RCAEval RE1, $N = 75$ each), across six configurations, the decomposition did not produce a statistically significant improvement over a single-agent baseline, including under a configuration designed specifically to test the architecture's own context-hygiene hypothesis. We report this as an honest, triangulated finding, not a result to explain away: it suggests multi-agent LLM decomposition's benefit for infrastructure triage is more setup-dependent than prior work (Flow-of-Action [34], MA-RCA [35]) has reported.

We also found, and report in full, that neither verbalized nor self-consistency-based LLM confidence discriminates correct from incorrect hypotheses on this task, which materially limits the safety guarantee the paper's confidence-gated autonomy mechanism can currently provide.

Finally, we tested a targeted extension beyond the base four-agent design: a Falsifier agent that adversarially validates the Reasoning agent's hypothesis before it reaches the Action agent, mirroring the validation mechanism recent multi-agent RCA systems report gains from. Unlike the base architecture, this extension produced a real, statistically significant effect — in both directions. It significantly improved accuracy on single-entity incidents (AIOps) and significantly degraded it on multi-service incidents (RCAEval), and tracing individual cases shows why: without service-dependency-graph awareness, the Falsifier mistakes propagated symptoms for root causes once an incident spans more than one service. This is the paper's clearest positive result, and we report its limits with the same rigor as its gains: verifying a hypothesis helps when the space of alternatives is well-scoped, and can actively hurt when it is not. A real service-topology fix (Section V-H) removes the RCAEval regression but does not yet turn it into a gain there— the honest current state, not rounded up. A control test further shows the AIOps gain does not require multi-agent decomposition: a single agent given the same Falsifier scores indistinguishably from the full multi-agent version ($p = 1.0$), consistent with concurrent evidence outside RCA [36], [37] (Section II-G). This paper's central claim is therefore not that multi-agent architecture adds value for LLM-based RCA, but that a specific *capability*—adversarial verification—does, independent of agent count.

All findings are reported because a verification or autonomy mechanism is only useful if its own failure modes are visible. The publicly available reference implementation and evaluation harness are designed to make all of them, and any future re-measurement, directly reproducible.

ACKNOWLEDGMENT

The author thanks the LangGraph and Google ADK open-source communities, and reviewers of early drafts of this work.

REFERENCES

[1] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly, 2016.
[2] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
[3] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.
[4] LangChain Team, "LangGraph: Building stateful, multi-actor applications with LLMs," 2024. [Online]. Available: https://github.com/langchain-ai/langgraph
[5] J. Zhao et al., "AIOps Challenge 2020: KPI anomaly detection," 2020. [Online]. Available: https://github.com/NetManAIOps/AIOps-Challenge-2020-Data
[6] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS*, 2022.
[7] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.
[8] S. Hong et al., "MetaGPT: Meta programming for multi-agent collaborative framework," arXiv:2308.00352, 2023.
[9] S. Patil et al., "Gorilla: Large language model connected with massive APIs," arXiv:2305.15334, 2023.
[10] C. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.
[11] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL*, 2019.
[12] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, 2021.
[13] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
[14] C. Lim, N. R. Murphy, and B. Beyer, "On-call and incident response," in *The Site Reliability Workbook*. O'Reilly, 2018.
[15] M. Bjorklund, "NETCONF Configuration Protocol," RFC 6241, IETF, 2011.
[16] B. Rabenstein and J. Volz, "Prometheus: A next-generation monitoring system," in *Proc. USENIX LISA*, 2015.
[17] H. Liu et al., "LogPrompt: Prompt engineering towards zero-shot and interpretable log analysis," arXiv:2308.07610, 2023.
[18] T. Winters, T. Manshreck, and H. Wright, *Software Engineering at Google*. O'Reilly, 2020.
[19] C. Jimenez et al., "SWE-bench verified," arXiv:2405.15793, 2024.
[20] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *ACM Queue*, vol. 14, 2016.
[21] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, 1947.
[22] J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, 1960.
[23] Y. Guo et al., "Exploring the feasibility of ChatGPT for event extraction," arXiv:2303.03836, 2023.
[24] X. Chen et al., "Automatic root cause analysis via large language models for cloud incidents," in *Proc. EuroSys*, 2024.
[25] W. Luo et al., "An unnamed knowledge graph approach to root-cause analysis for microservice systems," in *Proc. ISSRE*, 2022.
[26] D. Roy et al., "Exploring LLM-based agents for root cause analysis," arXiv:2403.04123, 2024.
[27] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.
[28] Meta AI, "Llama 3 model card," 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3/
[29] Elastic, "Elasticsearch: The official distributed search and analytics engine," 2024. [Online]. Available: https://www.elastic.co
[30] N. R. Murphy et al., "Embracing risk," in *Site Reliability Engineering*. O'Reilly, 2016, ch. 3.
[31] A. Basiri et al., "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.
[32] OpenTelemetry Authors, "OpenTelemetry specification," 2024. [Online]. Available: https://opentelemetry.io/docs/specs/
[33] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017.
[34] C. Pei, Z. Wang, F. Liu, Z. Li, Y. Liu, X. He, R. Kang, T. Zhang, J. Chen, J. Li, G. Xie, and D. Pei, "Flow-of-Action: SOP enhanced LLM-based multi-agent system for root cause analysis," in *Companion Proc. ACM Web Conference (WWW Companion '25)*, Sydney, Australia, 2025.
[35] F. Fu, H. Ding, Y. Qin, J. Yu, and D. Xu, "Leveraging multi-agent framework for root cause analysis," *Complex & Intelligent Systems*, vol. 12, no. 1, art. 4, 2026.
[36] J. Xu et al., "Rethinking the value of multi-agent workflow: a strong single agent baseline," arXiv:2601.12307, 2026.
[37] J. Su and C. Wu, "When do multi-agent systems outperform? Analysing the learning efficiency of agentic systems," arXiv:2602.08272, 2026.